DISEÑO DE PRODUCTOS Y SERVICIOS

CulturaAccesible Lima

Diseño e implementación del PMV1
Java · Python · MySQL

Semana 10 · Demostración académica
Modelo-Vista-Presentador, persistencia real y ciclo PDCA. Cartelera con fotos y fuentes; ejemplos académicos separados.

Cruz Gallardo, Loreley Itzayana

Hurtado Pacheco, Roy Kevin

Machuca García, Celso Eduardo

Molina Tapia, Eduardo Paco

Zambrano Canales, Esteban Angel

Sección 34971
Docente: Jorge Alfredo Guevara Jimenez
Actualizado: 18 de octubre de 2026 · Lima

Entrega técnica preparada a partir de los informes del equipo. Las funciones pendientes se identifican de forma explícita.

01 / Problema, alcance y equipo

Una persona que desea asistir a una actividad cultural necesita comprobar apoyos específicos antes de decidir. Una etiqueta general de «accesible» no demuestra que una función concreta tenga LSP, audiodescripción u otros recursos.

CulturaAccesible Lima organiza ese recorrido en descubrir, comprobar y coordinar. Esta entrega implementa principalmente descubrir y comprobar, con orientación de precios y reportes como extensiones de demostración.

Resultado funcional de esta entrega

Componente	Comportamiento comprobado
Catálogo	49 funciones con fotos y fuentes; 8 ejemplos ficticios separados.
Confianza	Estados textuales, alcance, fuente y fecha; evidencia ausente o vencida no confirma disponibilidad.
Persistencia	MySQL real; reportes con identificador y estado Pendiente.
Python	Estimación y clasificación sintéticas consultadas desde Java.

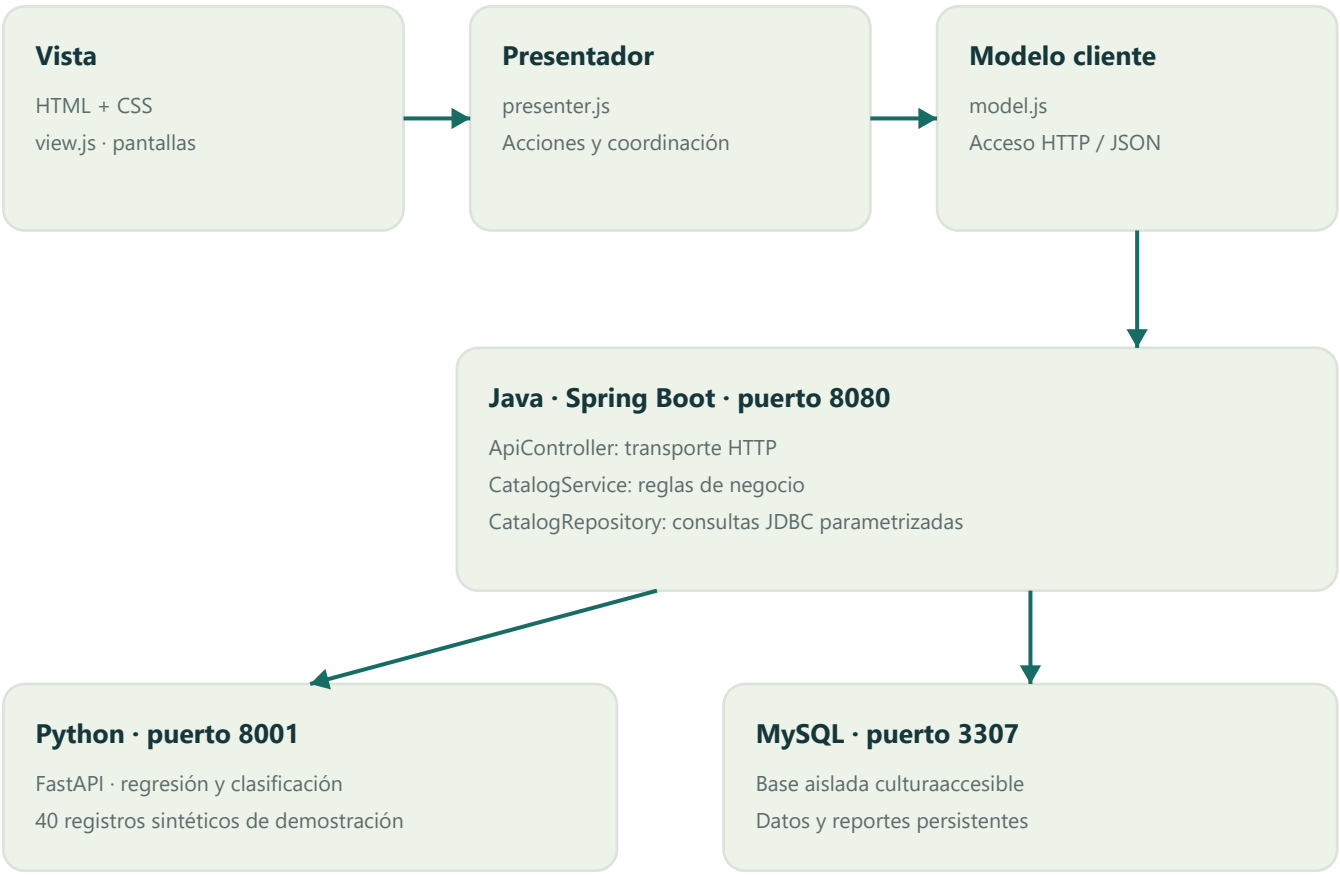
Distribución propuesta de roles

Rol	Asignación propuesta	Entrega
Arquitecto de Software	Loreley Itzayana Cruz Gallardo	Diseño, estructura MVP y contratos.
Diseñador de BBDD	Roy Kevin Hurtado Pacheco	Modelo, SQL e integridad.
Especialista FrontEnd	Celso Eduardo Machuca García	Vista, presentador y accesibilidad.
Especialista BackEnd	Eduardo Paco Molina Tapia	Java, conexión MySQL y Python.
Ingeniero de Calidad	Esteban Angel Zambrano Canales	Pruebas, evidencias y PDCA.

La asignación nominal es una propuesta organizativa y debe revisarse en el equipo. No acredita que esas personas hayan ejecutado individualmente las actividades descritas.

02 / Arquitectura implementada

Se aplica MVP a la interfaz web. La vista presenta, el presentador coordina acciones y el modelo cliente accede a servicios. Java conserva las reglas de negocio y MySQL los datos. Python atiende AccesI y la orientación de precios; ambos reciben solicitudes desde Java.

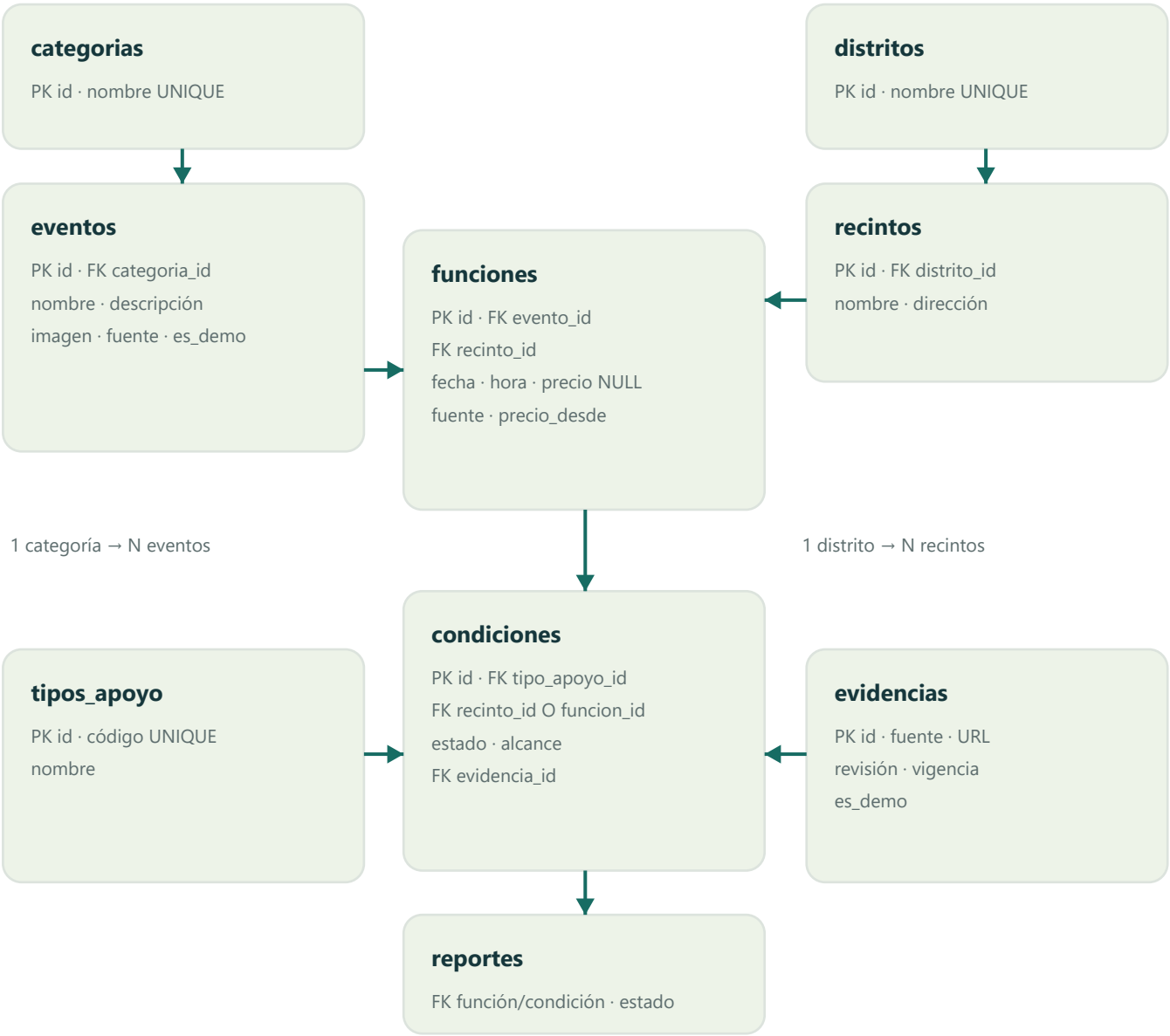


Responsabilidad	Archivo o componente
Vista	frontend/index.html, styles.css y js/view.js
Presentador	frontend/js/presenter.js
Modelo cliente	frontend/js/model.js
Negocio y persistencia	CatalogService y CatalogRepository
Integración Python	PricingService → POST /estimate; ChatService → POST /chat

La guía propone MVC o hexagonal. Se usa MVP por la elección comunicada por el equipo. Spring aporta transporte y servicios; su presencia por sí sola no demuestra MVP. Los archivos .puml documentan el diseño editable.

03 / Modelo de datos MySQL

Nueve tablas: categorías y distritos compartidos; eventos, recintos y funciones; apoyos, evidencias, condiciones y reportes. Evento describe la actividad; función fija fecha, hora y recinto. Las condiciones distinguen infraestructura del recinto y apoyos de una función. Las evidencias pueden respaldar varias declaraciones.



Reglas de integridad

El diagrama resume relaciones principales; el modelo .mwb y el SQL detallan las diez claves foráneas. Precio NULL significa no informado; cero significa ingreso gratuito. Los CHECK impiden precios negativos, estados ajenos al catálogo y condiciones asociadas simultáneamente a recinto y función. Las claves foráneas preservan relaciones y los UNIQUE evitan repetir el mismo apoyo para el mismo alcance.

02_demo.sql contiene ocho ejemplos ficticios. CulturaAccesibleLima.sql carga la cartelera y los ejemplos en una base nueva. El inicio migra bases existentes con respaldo privado; no elimina filas.

04 / Selección de herramientas

Escala de 1 a 5. Total ponderado = suma de peso × puntaje / 100. Son valoraciones académicas propuestas para este contexto, no resultados de una comparación experimental.

IDE

Factor	Peso	Visual Studio Code	IntelliJ IDEA
Compatibilidad Java/Python	30%	5	4
Accesibilidad del entorno existente	20%	5	5
Facilidad de uso	20%	4	4
Trabajo con Git	15%	5	5
Recursos del equipo	15%	4	3
Total	100%	4.65	4.20

Selección: **Visual Studio Code**.

Framework Java

Factor	Peso	Spring Boot	Javalin
Reglas y mantenibilidad	25%	5	4
Validación y pruebas	25%	5	3
Integración MySQL	20%	5	4
Aprendizaje	20%	3	4
Tiempo inicial	10%	3	4
Total	100%	4.40	3.75

Selección: **Spring Boot**.

Servicio Python

Factor	Peso	FastAPI	Flask
Validación de entradas	30%	5	3
Documentación de API	20%	5	3
Uso con modelos Python	20%	5	5
Facilidad de implementación	20%	4	5
Mantenibilidad	10%	4	4
Total	100%	4.70	3.90

Selección: **FastAPI**.

05 / Datos y organización

Escala de 1 a 5. Total ponderado = suma de peso × puntaje / 100. Son valoraciones académicas propuestas para este contexto, no resultados de una comparación experimental.

Base de datos

Factor	Peso	MySQL	PostgreSQL
Instalación disponible	25%	5	2
Integridad relacional	25%	5	5
Conexión desde Java/Python	25%	5	5
Aprendizaje del equipo	15%	4	4
Accesibilidad para el piloto	10%	5	5
Total	100%	4.85	4.10

Selección: **MySQL**.

Organización

Factor	Peso	Trello	GitHub Projects
Elección previa del equipo	30%	5	2
Facilidad del tablero	25%	5	4
Vinculación con repositorio	20%	3	5
Acceso para el equipo	15%	5	5
Claridad de estados	10%	5	4
Total	100%	4.60	3.75

Selección: **Trello**.

Decisión de implementación

Java y Python responden a tareas distintas. Spring Boot maneja catálogo y persistencia; FastAPI expone la PoC de precios. Se usa JDBC para consultas parametrizadas explícitas. MySQL se aprovecha como instalación existente, con una instancia aislada en el puerto 3307. Trello organiza tarjetas y GitHub queda previsto para revisión del código.

Trello y GitHub no fueron modificados desde esta sesión. Se entregan un backlog CSV y un proyecto preparado para publicar en el repositorio del equipo.

Historias / 01 a 03

Base documental: 10 historias del APF2 + 2 de IA en la semana 9 = 12. Ampliación del 50%: 6 adicionales. Total consolidado: 18. Las historias de IA se renumeran HU11 y HU12.

HU01 · Buscar eventos

Etapa 1 · Prioridad Alta · **Terminado: verificación técnica local**

Como visitante, quiero buscar eventos por nombre o disciplina para descubrir actividades que me interesen.

CA1. Dado un término presente en título, disciplina o recinto, cuando busco, entonces obtengo las funciones coincidentes, ignorando diferencias de tildes y mayúsculas.

CA2. Dado un término sin coincidencias, cuando busco, entonces aparece un mensaje de cero resultados y puedo limpiar la búsqueda.

Evidencia o pendiente. Java: accentsDoNotBlockSearch, noMatchesReturnsEmpty. Integración 12-14. Navegador.

HU02 · Combinar filtros

Etapa 1 · Prioridad Alta · **Terminado: verificación técnica local**

Como visitante, quiero filtrar por fecha, distrito, categoría y precio para reducir opciones relevantes.

CA1. Dado que selecciono varios filtros, cuando busco, entonces cada resultado cumple todos los filtros seleccionados.

CA2. Dado un precio no informado, cuando filtro ingreso gratuito, entonces esa función queda excluida; al limpiar recupero el catálogo.

Evidencia o pendiente. Java: filtersCombine, unknownPriceIsNotFree, zeroPriceIsFree. Integración 03-04. Navegador.

HU03 · Abrir ficha por función

Etapa 1 · Prioridad Alta · **Terminado: verificación técnica local**

Como persona con necesidades de accesibilidad, quiero abrir una ficha por función para comprobar apoyos concretos.

CA1. Dada una tarjeta, cuando la abro, entonces la ficha identifica la misma función, recinto, fecha y hora.

CA2. Dado que cierro la ficha, cuando regreso al catálogo, entonces se conservan los filtros; un identificador inexistente devuelve 404.

Evidencia o pendiente. Integración 07-09. Navegador: abrir/cerrar y enlace directo.

Historias / 04 a 06

Base documental: 10 historias del APF2 + 2 de IA en la semana 9 = 12. Ampliación del 50%: 6 adicionales. Total consolidado: 18. Las historias de IA se renumeran HU11 y HU12.

HU04 · Estados con fuente y fecha

Etapa 1 · Prioridad Alta · **Terminado: verificación técnica local**

Como visitante, quiero ver Disponible, No disponible o No confirmado con fuente y fecha para saber qué puedo dar por verificado.

CA1. Dada evidencia vigente y de alcance correcto, cuando abro la ficha, entonces leo estado textual, alcance, fuente y fecha.

CA2. Dada evidencia ausente, vencida o insuficiente para la función, cuando consulto, entonces el estado es No confirmado, sin inferir disponibilidad.

Evidencia o pendiente. Java: missingEvidenceCannotConfirm, expiredEvidenceCannotConfirm, venueLspDoesNotImplySessionSupport. Integración 07-08.

HU05 · Navegación accesible

Etapa 1 · Prioridad Alta · **En pruebas**

Como persona que usa teclado o lector de pantalla, quiero recorrer búsqueda y ficha sin ratón para decidir de forma autónoma.

CA1. Dado el uso de teclado, cuando recorro controles y diálogos, entonces hay etiquetas, foco visible y cierre con Escape que devuelve el foco.

CA2. Dado el uso de lector de pantalla, cuando cambian los resultados, entonces se anuncia el recuento. Debe probarse con una tecnología de apoyo real antes de cerrar la historia.

Evidencia o pendiente. Revisión parcial en navegador: etiquetas, diálogo y tamaño 390×844. Falta lector de pantalla y auditoría completa.

HU06 · Consultar un apoyo indispensable

Etapa 2 · Prioridad Alta · **Por hacer**

Como persona que requiere un apoyo, quiero indicar cuál es indispensable y enviar una consulta escrita para obtener respuesta.

CA1. Dada una función y consentimiento explícito, cuando envío una consulta válida, entonces obtengo un identificador y estado Enviada.

CA2. Dado que falta consentimiento o contenido obligatorio, cuando envío, entonces se rechaza la consulta sin confirmarla.

Evidencia o pendiente. Criterios propuestos; no hay implementación ni prueba de envío real.

Historias / 07 a 09

Base documental: 10 historias del APF2 + 2 de IA en la semana 9 = 12. Ampliación del 50%: 6 adicionales. Total consolidado: 18. Las historias de IA se renumeran HU11 y HU12.

HU07 · Seguimiento de solicitudes

Etapa 2 · Prioridad Alta · **Por hacer**

Como visitante, quiero consultar el estado y el historial de mi solicitud para no confundir envío con confirmación.

CA1. Dada una solicitud propia, cuando consulto su historial, entonces aparecen cambios con fecha, responsable y apoyo específico.

CA2. Dado un envío sin respuesta, cuando reviso, entonces no se muestra como Confirmada; otra persona no puede consultar mi solicitud sin autorización.

Evidencia o pendiente. Criterios propuestos; requiere identidad, permisos y módulo de solicitudes.

HU08 · Gestión del organizador

Etapa 2 · Prioridad Alta · **Por hacer**

Como organizador, quiero actualizar condiciones por sesión y responder solicitudes para mantener información vigente.

CA1. Dado un organizador autorizado, cuando actualiza una función o responde, entonces quedan evidencia, fecha y responsable.

CA2. Dado un usuario sin permiso, cuando intenta editar, entonces se rechaza la operación y no cambia la ficha.

Evidencia o pendiente. Criterios propuestos; no se entrega panel de organizadores.

HU09 · Preferencias opcionales

Etapa 3 · Prioridad Media · **Por hacer**

Como visitante frecuente, quiero guardar preferencias de forma opcional para encontrar eventos afines más rápido.

CA1. Dado consentimiento opcional, cuando guardo preferencias funcionales, entonces puedo recuperarlas y modificarlas.

CA2. Dado que no deseo guardarlas o que las elimino, cuando vuelvo al catálogo, entonces puedo buscar sin registro obligatorio.

Evidencia o pendiente. Criterios propuestos; no se almacenan preferencias ni diagnósticos en esta versión.

Historias / 10 a 12

Base documental: 10 historias del APF2 + 2 de IA en la semana 9 = 12. Ampliación del 50%: 6 adicionales. Total consolidado: 18. Las historias de IA se renumeran HU11 y HU12.

HU10 · Comparar eventos

Etapa 3 · Prioridad Media · **Por hacer**

Como visitante, quiero comparar eventos y recibir explicaciones de coincidencia para elegir con menos esfuerzo.

CA1. Dadas varias funciones, cuando las comparo, entonces veo diferencias de precio, fecha y apoyos con sus fuentes.

CA2. Dado un apoyo No confirmado, cuando se explica una coincidencia, entonces la explicación mantiene esa incertidumbre.

Evidencia o pendiente. Criterios propuestos; el filtro de apoyos implementado no equivale al comparador.

HU11 · Precio orientativo

Etapa 1 · Prioridad Media · **Terminado: verificación técnica local**

Como visitante, quiero consultar un precio orientativo según categoría, distrito y mes para planificar mi presupuesto y verificar la fuente oficial.

CA1. Dados datos dentro de la cobertura, cuando consulto desde Java, entonces Python devuelve una estimación finita y no negativa con aviso de datos sintéticos.

CA2. Dado un mes decimal, booleano o fuera de rango, cuando consulto, entonces no se produce una predicción. El precio oficial conserva prioridad.

Evidencia o pendiente. Python: estimación válida y entradas inválidas. Integración 15-19. Nueva PoC, distinta a la del PDF anterior.

HU12 · Clasificar rango de precio

Etapa 2 · Prioridad Media · **Terminado: verificación técnica local**

Como visitante que explora entradas con un límite de referencia de S/ 40, quiero consultar el rango estimado de un evento sin precio publicado para priorizar la verificación de su precio oficial.

CA1. Dados datos válidos, cuando solicito orientación, entonces recibo Hasta S/ 40 o Mayor de S/ 40 como clasificación independiente y de demostración.

CA2. Dado que regresión y clasificación discrepan, cuando se muestran, entonces se advierte la discrepancia y no se garantiza el presupuesto.

Evidencia o pendiente. Python y consulta web. Esta extensión se adelanta como demostración, sin cambiar el alcance original de las etapas.

Historias / 13 a 15

Base documental: 10 historias del APF2 + 2 de IA en la semana 9 = 12. Ampliación del 50%: 6 adicionales. Total consolidado: 18. Las historias de IA se renumeran HU11 y HU12.

HU13 · Filtrar apoyos indispensables

Etapa 1 · Prioridad Alta · **Terminado: verificación técnica local**

Como visitante, quiero filtrar eventos por los apoyos de accesibilidad que considero indispensables para encontrar funciones compatibles con mis necesidades.

CA1. Dados varios apoyos elegidos, cuando aplico el filtro, entonces cada resultado tiene todos esos apoyos en Disponible para el alcance válido.

CA2. Dado un apoyo ausente o vencido, cuando filtro por él, entonces la función queda excluida y se informa si no hay coincidencias.

Evidencia o pendiente. Java: supportsRequireAvailable. Integración 05-06. Navegador con filtro LSP.

HU14 · Compartir una ficha

Etapa 1 · Prioridad Media · **Terminado: verificación técnica local**

Como visitante, quiero compartir un enlace directo a una función y su ficha de accesibilidad para coordinar mi asistencia con un acompañante.

CA1. Dada una ficha, cuando copio y abro su enlace, entonces se abre la misma función identificada por su ID.

CA2. Dado que el portapapeles no está disponible, cuando intento copiar, entonces aparece un campo seleccionable con el enlace; se explica que requiere acceso a esta aplicación.

Evidencia o pendiente. Navegador: enlace profundo y recarga. Copia mediante API de portapapeles con alternativa manual.

HU15 · Reportar información incorrecta

Etapa 2 · Prioridad Media · **Terminado: verificación técnica local**

Como visitante, quiero reportar información de accesibilidad incorrecta o desactualizada para que el equipo pueda revisarla.

CA1. Dado un comentario válido, cuando envío el reporte, entonces MySQL lo guarda con identificador y estado Pendiente.

CA2. Dado un comentario inválido o fallo de persistencia, cuando envío, entonces no aparece confirmación de guardado; el reporte no modifica automáticamente la ficha.

Evidencia o pendiente. Integración 20-22 y consulta directa a MySQL. Reporte de prueba visible en navegador.

Historias / 16 a 18

Base documental: 10 historias del APF2 + 2 de IA en la semana 9 = 12. Ampliación del 50%: 6 adicionales. Total consolidado: 18. Las historias de IA se renumeran HU11 y HU12.

HU16 · Revisar evidencias

Etapa 2 · Prioridad Alta · **Por hacer**

Como revisor de la plataforma, quiero comprobar las evidencias de cada condición antes de aprobar su publicación para mantener información confiable.

CA1. Dado un revisor autorizado y evidencia suficiente, cuando aprueba una condición, entonces se registran responsable, fecha, alcance y resultado.

CA2. Dada evidencia insuficiente o permiso ausente, cuando se intenta aprobar, entonces la condición no se publica como Disponible.

Evidencia o pendiente. Criterios propuestos. La degradación automática de estados sí funciona; el flujo de aprobación con permisos queda pendiente.

HU17 · Avisos de cambios

Etapa 3 · Prioridad Media · **Por hacer**

Como visitante, quiero recibir avisos de cambios de fecha, cancelación o apoyos de una función que sigo para reorganizar mi visita.

CA1. Dada una suscripción consentida, cuando cambia información relevante de la función, entonces recibo un aviso que identifica el cambio.

CA2. Dado que retiro mi suscripción, cuando ocurre otro cambio, entonces no se envía una nueva notificación.

Evidencia o pendiente. Criterios propuestos; no se envían correos, mensajes ni notificaciones en esta entrega.

HU18 · Búsqueda en lenguaje natural limitado

Etapa 1 · Prioridad Media · **Terminado: reglas limitadas verificadas**

Como visitante, quiero consultar el catálogo mediante lenguaje natural para encontrar eventos sin conocer todos los filtros disponibles.

CA1. Dada una frase con categorías, distritos o apoyos reconocidos, cuando la envío, entonces se proponen filtros visibles que puedo cambiar.

CA2. Dada una frase sin filtros reconocidos, cuando la envío, entonces se explica el límite y se permite usar la búsqueda manual sin inventar eventos.

Evidencia o pendiente. Navegador: Teatro en Miraflores con LSP. Implementación por reglas, sin IA generativa ni comprensión general del lenguaje.

Implementación / recorrido completo

Paso	Comportamiento
1. Buscar	El presentador recoge criterios. El modelo solicita GET /api/functions.
2. Consultar	Java valida los filtros y consulta MySQL; combina criterios mediante reglas de negocio.
3. Presentar	La vista muestra tarjetas, recuento accesible y estados vacíos o de error.
4. Comprobar	La ficha identifica una función y desglosa los siete apoyos con su evidencia.
5. Estimar	Java solicita a Python una orientación cuando el usuario lo decide; el precio oficial conserva prioridad.
6. Reportar	Java valida e inserta el reporte en MySQL; solo después devuelve su identificador.

Estructura del proyecto

backend/ · Java, Maven y pruebas de negocio
frontend/ · HTML, CSS y módulos MVP
python-service/ · FastAPI, modelos y conjunto sintético
database/ · esquema y semilla SQL
scripts/ · inicio, compilación e integración
docs/ · informe, historias, matrices y evidencias
app/ · ejecutable Java con la interfaz incluida

Tratamiento del fallo

La aplicación distingue entradas inválidas, datos sin cobertura y servicios no disponibles. La caída de Python afecta a la orientación de precios y conserva el recorrido del catálogo. Un error de persistencia no se presenta como reporte guardado.

La ayuda en lenguaje natural es limitada y usa reglas locales. Puede reconocer categorías, distritos, apoyos y algunos filtros temporales; permite revisar los filtros y no inventa actividades.

Verificación / resultados ejecutados

Capa	Pruebas	Resultado
Java: reglas de catálogo	20	Aprobadas
Python: API, precios y Accesl	43	Aprobadas
Integración HTTP + MySQL	39	Aprobadas
Migración e instalación de MySQL	9	Aprobadas
Fallos controlados de servicios	4	Aprobadas
Navegador	Revisión manual	Búsqueda, fichas, reportes, orientación y tamaño móvil

Las pruebas de integración se ejecutaron contra Spring Boot, FastAPI y MySQL reales. Incluyen consulta de la fila insertada, filtros combinados, rechazo de precios negativos por MySQL, fuente vencida, errores de cobertura y rechazo de meses decimales o booleanos.

Métricas de la nueva PoC sintética

Medida	Resultado
Registros / entrenamiento / prueba	40 / 30 / 10
MAE de regresión	S/ 3.427
Accuracy de clasificación	0.9
F1 de la clase Hasta S/ 40	0.909
Semilla	42

La implementación utiliza regresión lineal y árbol de decisión predefinidos; no se presenta una selección comparativa nueva de modelos. Los resultados corresponden a registros sintéticos nuevos y no reproducen las cifras del informe anterior.

Ninguna métrica demuestra precisión en precios reales. La regresión y el clasificador son independientes y pueden discrepar. La interfaz muestra un aviso explícito en ese caso. La revisión técnica tampoco sustituye pruebas con personas usuarias ni una auditoría WCAG completa.

Mejora continua / PDCA

Cada mejora enlaza una necesidad o riesgo, la corrección implementada y la evidencia que permite verificarla.

Ciclo	Planificar	Hacer	Verificar y actuar
Confianza	Evitar apoyos disponibles sin respaldo suficiente.	Degradar evidencia ausente, vencida o de alcance incorrecto.	Reglas Java y filtro LSP sobre ejemplos. Conservar casos de regresión.
Entradas	Evitar convertir un mes decimal en entero.	Rechazar coerción en Java y usar StrictInt en Python.	Mes 6.5, booleano y fuera de rango rechazados. Mantener validación en ambas API.
Precio	Explicar resultados que discrepan entre modelos.	Agregar models_disagree y mensaje visible.	Danza/Miraflores/octubre: S/ 49.98 y Hasta S/ 40. Se advierte que no confirma presupuesto.
Instalación	Admitir punto y coma dentro de textos SQL.	Ejecutar el archivo con soporte de sentencias múltiples y semilla idempotente.	Inicialización y recarga sin duplicados. Mantener inicio reproducible.

Seguimiento del trabajo

Trello se organiza en Por hacer, En desarrollo, En pruebas y Terminado. Cada tarjeta incluye historia, criterios, prioridad y evidencia. HU05 permanece En pruebas porque falta validación con lector de pantalla. Las solicitudes, permisos, preferencias, comparación y notificaciones permanecen Por hacer.

Git y revisión

El código se prepara en un repositorio local. Para el siguiente incremento se propone una rama feature/nombre, pruebas, commit y pull request. La publicación en GitHub depende del repositorio del equipo y no se presenta como realizada.

Evidencias / interfaz y datos

Capturas de la implementación local. Los originales se incluyen en docs/evidencias para inspección a tamaño completo.



Figura 1. Cartelera con fotografía, crédito y fecha de recuperación.

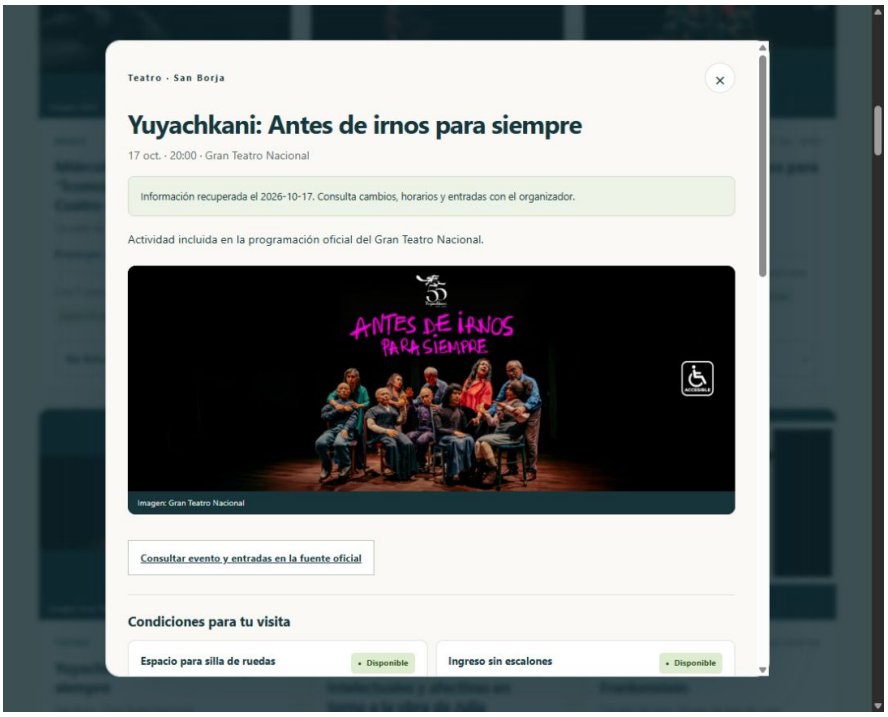


Figura 2. Ficha de una función con estados, alcance y fechas de revisión.

Evidencias / orientación y móvil

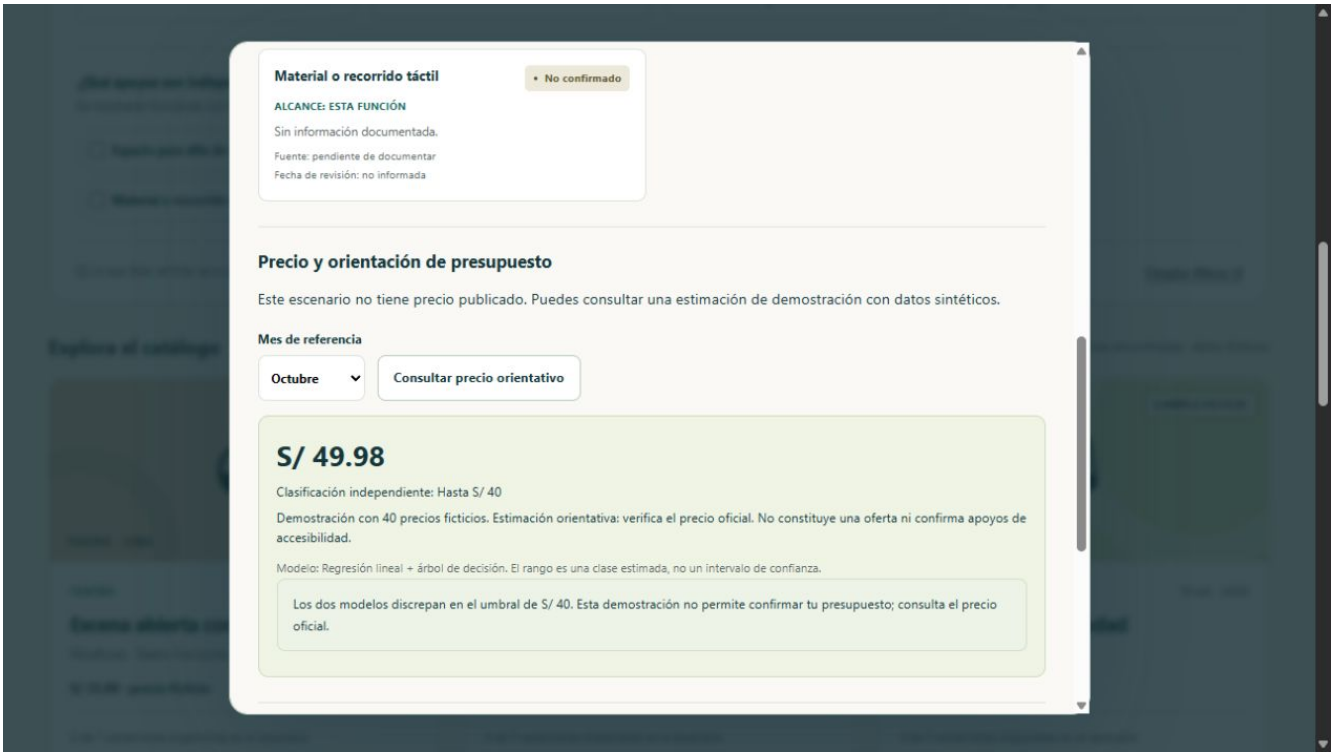


Figura 3. Orientación solicitada a Python y aviso de discrepancia entre modelos.

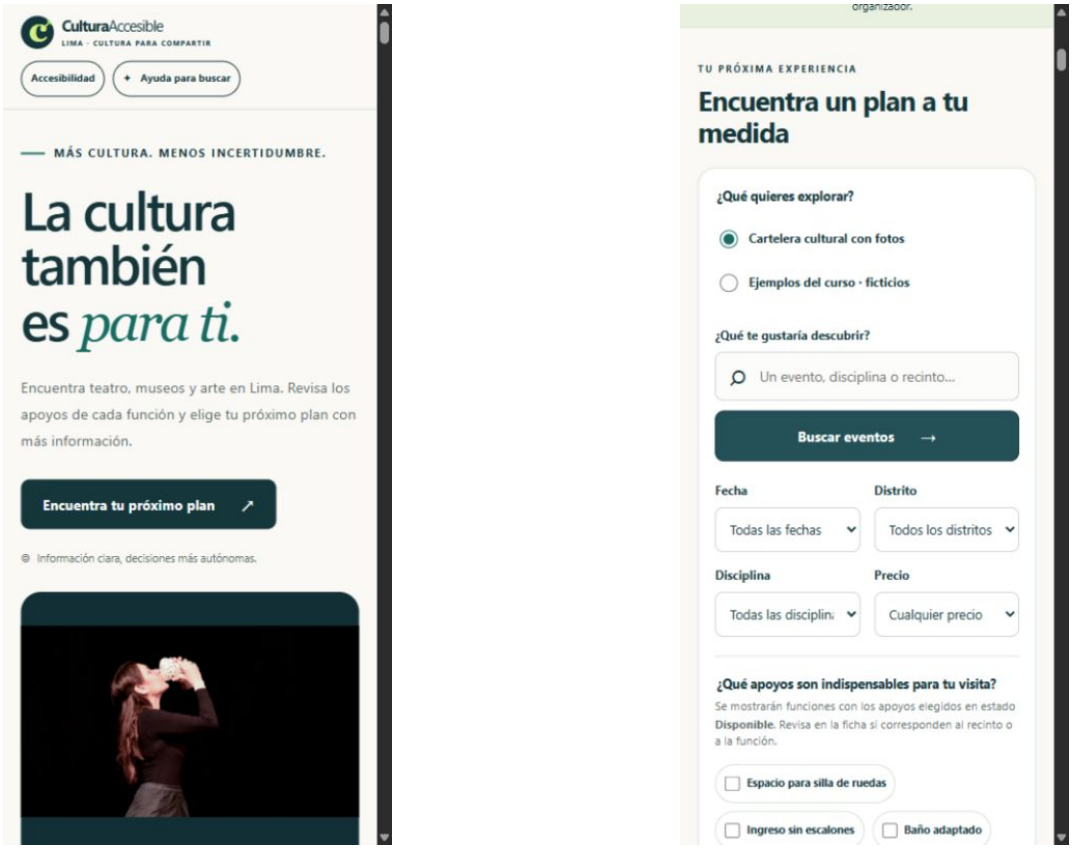


Figura 4. Revisión a 390 × 844: encabezado y filtros adaptados; sin desbordamiento horizontal observado.

Instalación / demostración y entrega

Preparación

Extraer todo el ZIP. Requisitos: Java 17 o posterior, Python 3.11/3.12 y MySQL Server 8.0.16 o posterior. La primera ejecución requiere Internet para instalar las dependencias de Python fijadas en requirements-lock.txt.

Inicio

Abrir **Abrir-demo.cmd**. El programa detecta las herramientas habituales y crea una instancia MySQL aislada. Esperar el mensaje de aplicación lista y abrir **http://127.0.0.1:8080**. Mantener la ventana abierta. Ctrl+C detiene los componentes de esta demostración.

Servicio	Dirección / puerto
Java y web	127.0.0.1:8080
Python	127.0.0.1:8001
MySQL de demostración	127.0.0.1:3307
Credenciales y datos locales	.local/ · excluido del paquete y de Git

Guion breve para exponer

1. Mostrar la cartelera con fotos y abrir una fuente oficial.
2. Cambiar a ejemplos del curso y combinar distrito, disciplina y LSP.
3. Abrir una ficha y distinguir recinto, función y No confirmado.
4. Consultar Danza en movimiento para demostrar Java → Python.
5. Registrar un reporte y verificar su identificador en MySQL.
6. Mostrar pruebas, diagrama y registro PDCA.

Compilar y probar

Abrir backend/pom.xml con IntelliJ o la carpeta completa con VS Code. Detener la aplicación antes de compilar. scripts/Compilar.ps1 ejecuta las pruebas Java y actualiza el JAR. Con la demo iniciada, scripts/Probar.ps1 ejecuta Python y las pruebas de integración.

README.md explica rutas alternativas y el uso manual de una base MySQL propia. No compartir .local/runtime.json ni publicar contraseñas. El sitio ya publicado no fue modificado por esta entrega.

Referencias / procedencia y límites

Los documentos previos aportan necesidad, propuesta de valor, etapas e historias. El código y las métricas de esta entrega corresponden a una implementación local con cartelera recuperada y ejemplos sintéticos separados.

Material del equipo

APF2, semana 8: apartados 2.3 y 2.4, páginas 13-18. Base de diez historias y alcance por etapas.
Actividad 06: página 4. Propuesta de valor e información por función.
Informe de semana 9: páginas 2 y 13. Historias de estimación y clasificación de precios.
Guía de trabajo 10: páginas 1 y 8. Diseño, implementación, pruebas y PDCA.

Documentación técnica

Spring: Acceso a MySQL con Spring
FastAPI: Primeros pasos y operaciones HTTP
MySQL: Restricciones CHECK
scikit-learn: Evaluación de modelos

Referencia funcional

Sitio publicado de CulturaAccesible Lima. Se inspeccionó como referencia; no se cambió su publicación ni se utilizó como prueba de que Java, Python o MySQL ya estuvieran integrados.

Trabajo pendiente

Integración con organizadores reales, actualización del catálogo, identidad y permisos, solicitudes de apoyo, seguimiento, revisión con responsables, preferencias, comparación y avisos. También faltan auditoría completa de accesibilidad, lectores de pantalla y pruebas con personas usuarias.

No se inventan entrevistas, aprobaciones de organizadores, confirmaciones de accesibilidad, publicaciones en GitHub/Trello ni aportes individuales. La revisión móvil y de teclado descrita es parcial y no certifica conformidad WCAG.

Actualización / cartelera y accesibilidad

Se recuperaron 49 funciones de la web publicada del equipo el 17/10/2026. Se conserva el título, la fecha, la imagen con su crédito, el enlace oficial y las declaraciones visibles de condiciones. No se presenta esta importación como una comprobación independiente de las 49 actividades.

Comportamiento de la nueva versión

El catálogo principal muestra la cartelera con fotos. Un selector separado permite explorar los ocho ejemplos ficticios para las pruebas del curso. Los horarios desconocidos permanecen NULL y los precios de partida conservan «Desde». La fuente oficial permite consultar cambios y entradas.

El panel de accesibilidad permite texto al 100%, 112,5% y 125%, alto contraste, subrayado de enlaces y reducción de movimiento. Las preferencias se guardan en este navegador. Estas funciones no acreditan conformidad WCAG ni sustituyen pruebas con lectores de pantalla.

Datos, editor y verificación

MySQL conserva las 57 funciones, las evidencias y los reportes. La migración agrega campos de procedencia e imagen sin eliminar datos. La importación repetida fue probada sin duplicados. VS Code incorpora un espacio de trabajo y tareas de inicio, compilación y pruebas.

Verificación	Resultado
Java / Python / integración / fallos / migración	20 / 43 / 39 / 4 / 9 aprobadas: 115 en total
MySQL	127.0.0.1:3307 · culturaaccesible · cal_app
Ventana de Workbench	Conexión CulturaAccesible Lima guardada y comprobada el 18/10/2026; esquema y resultados visibles.

La instantánea no se actualiza automáticamente. Las imágenes se cargan desde sus sitios de origen y requieren conexión. El sitio publicado original no fue modificado. Fuentes contrastadas: MALI, actividad El camino de Nora y página de visitas; Gran Teatro Nacional, Yuyachkani: Antes de irnos para siempre.

Accesl / conversación conectada

El chat de la web local responde a saludos y consulta actividades por mes, día, presupuesto y apoyos. Conserva criterios entre mensajes: «noviembre» y después «y los más baratos» mantienen el periodo. «El 18 de octubre» se aplica como un día exacto, no como todo el mes.

Recorrido y modelo

Página → Java → MySQL → Java → Python → respuesta. Java obtiene la cartelera y las condiciones efectivas. Python clasifica la intención con TF-IDF y regresión logística sobre ejemplos académicos; reglas de fechas, filtros y orden construyen la respuesta. No se usa un modelo generativo ni se ejecuta SQL del usuario. No se afirma una tasa de precisión validada con personas usuarias.

Los resultados enlazan la ficha y la fuente oficial. Para comparar precios se excluyen importes desconocidos; «Desde» conserva su significado de tarifa mínima. No se usan predicciones sintéticas para afirmar precios oficiales o disponibilidad de apoyos.

Comparación con la referencia

Se probaron «hola», una consulta de noviembre y otra por precio con el día 18 de octubre en el sitio original del equipo. La interfaz respondió al saludo y a mes/precio; en la tercera consulta devolvió eventos de otros días del mes. La versión local tiene pruebas que exigen coincidencia con el día exacto. No se inspeccionó el código interno del sitio de referencia.

Las conversaciones no se almacenan. Los criterios viven en memoria de la pestaña. Nueva conversación o cambio de catálogo limpia el contexto. Una caída del asistente devuelve un aviso y permite seguir consultando el catálogo.

Accesl / evidencia de uso

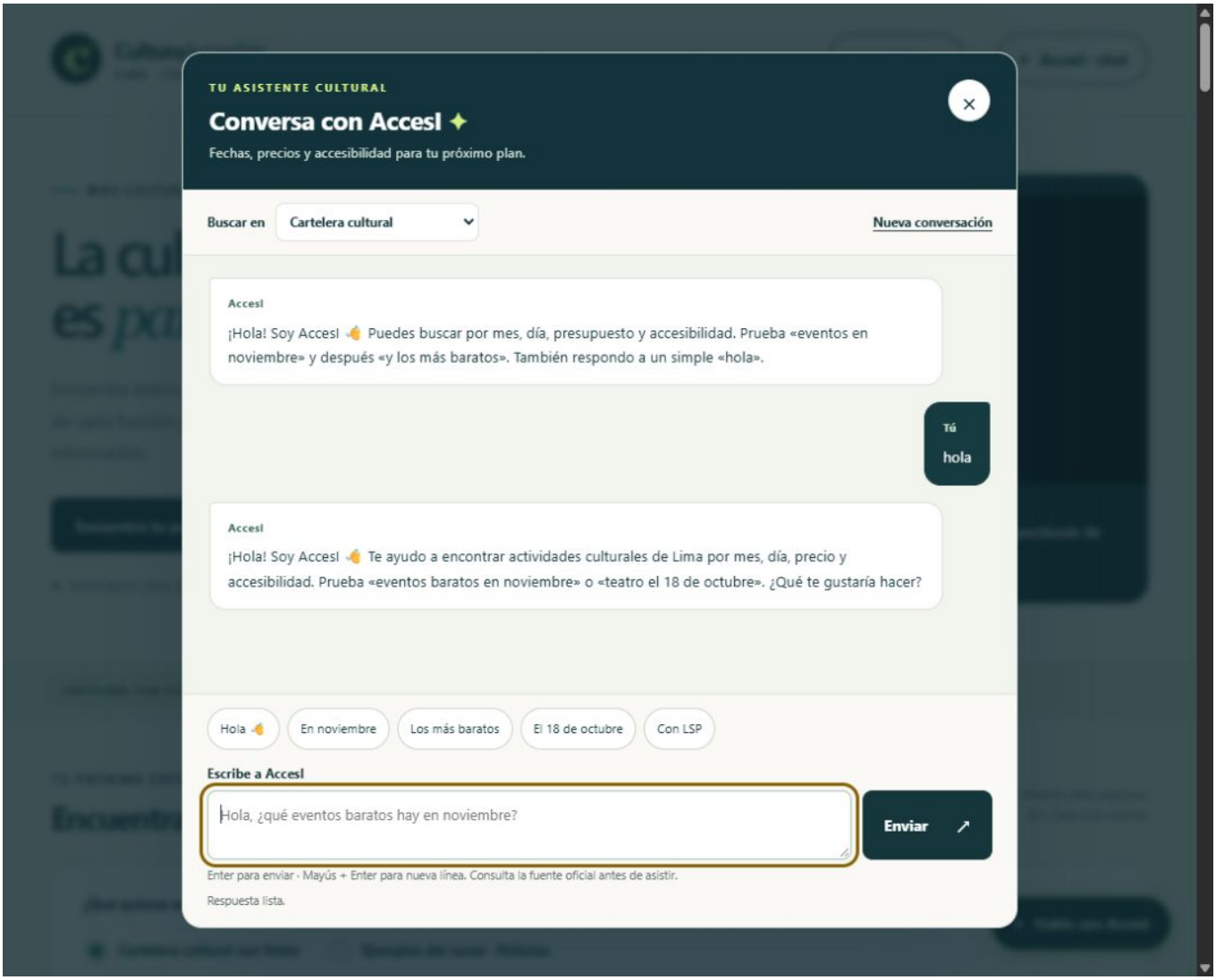


Figura 5. Saludo real desde la página local, recibido de Java y Python.

Validación de esta versión

115 pruebas aprobadas: 20 de Java, 43 de Python, 39 de integración, 4 de fallos y 9 de migración e instalación. Se verificaron saludo, memoria de mes, día exacto, presupuesto, separación de ejemplos, contexto inválido y caída del asistente. Revisión de móvil a 390 × 844 con texto al 125%, envío por teclado y ausencia de desbordamiento horizontal.

LEEME_PRIMERO.md explica el proyecto para la exposición y las rutas de edición. VS Code incluye tareas y perfil CMD para nuevos terminales. MySQL de la aplicación sigue conectado en 127.0.0.1:3307. Workbench ya está conectado a la misma base; el 18/10/2026 se verificaron el puerto, el usuario, las siete tablas originales y los datos mediante consultas SELECT. La normalización posterior amplía el esquema a nueve tablas y conserva los registros.

MySQL / conexión verificada

Verificación realizada el 18 de octubre de 2026. La conexión CulturaAccesible Lima quedó guardada y abierta en MySQL Workbench. Se ejecutaron consultas SELECT y se observaron los resultados en el editor SQL.

Comprobación	Resultado observado
Servidor y puerto	127.0.0.1:3307
Base y usuario	culturaaccesible · cal_app@127.0.0.1
Tablas	Captura anterior: 7. Esquema actual: 9; agrega categorias y distritos.
Funciones	57 en total: 49 de cartelera y 8 ejemplos ficticios
Página y Java	API de salud: ok; catálogo principal: 49 funciones

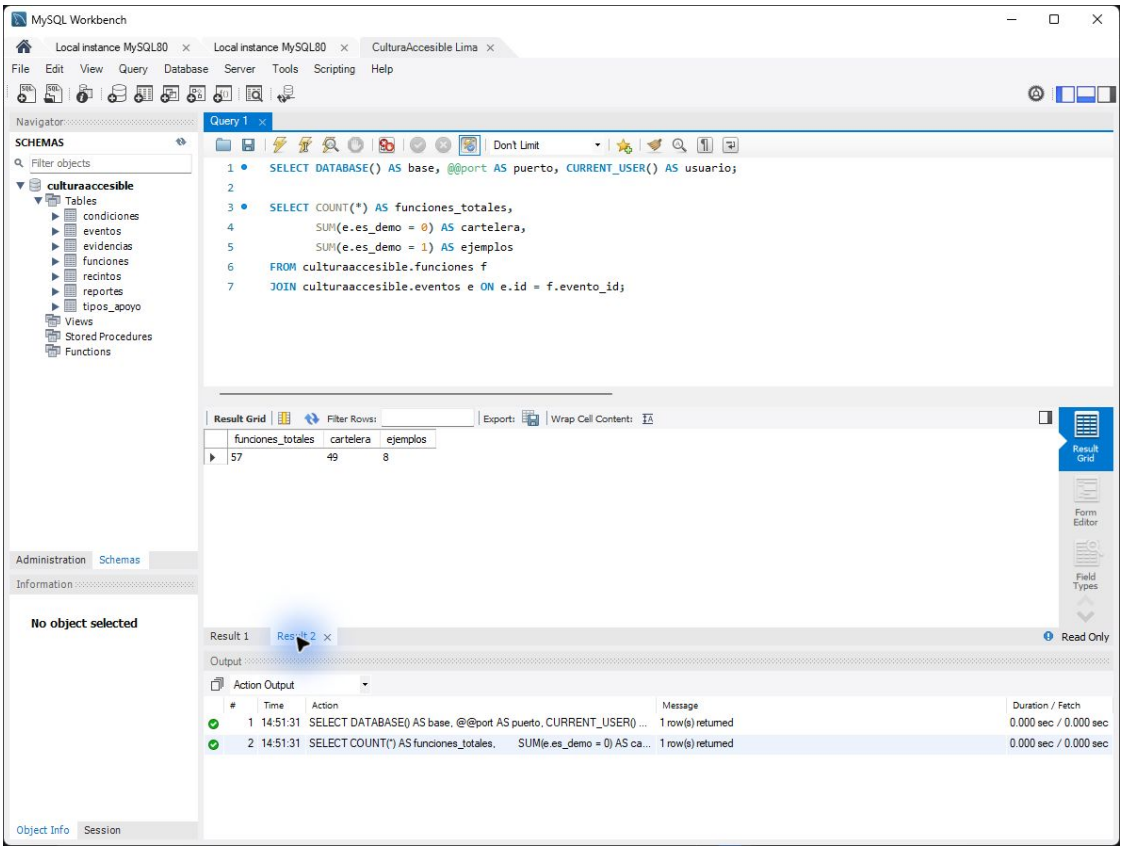


Figura 6. Workbench conectado: tablas del esquema, consulta SQL y recuento 57 / 49 / 8.

Esta comprobación confirma la conexión y los datos guardados. La cartelera sigue siendo la instantánea recuperada el 17/10/2026; no se declara una actualización automática ni una nueva validación de todos los eventos. No se incluyen contraseñas. Evidencias originales: mysql-workbench-conexion.jpg, mysql-workbench-datos.jpg y mysql-workbench.json.

MySQL / estructura mejorada

Versión 1.3: nueve tablas. Categorías y distritos tienen identificadores y nombres únicos; cada evento y recinto los referencia mediante una clave foránea obligatoria. Esto evita repetir los nombres en todos los registros y permite mantenerlos desde un catálogo común.

Datos conservados

Se guardó un respaldo SQL privado antes de migrar. La comparación lógica completa produjo el mismo hash antes y después: 57 eventos, 57 funciones, 6 recintos, 7 apoyos, 271 evidencias, 288 condiciones y los 5 reportes existentes se conservaron. Los IDs de eventos y funciones no cambiaron, por lo que los enlaces siguen funcionando.

Archivo	Uso
docs/CulturaAccesibleLima.mwb	Abrir en Workbench para editar las 9 tablas y 10 relaciones.
database/CulturaAccesibleLima.sql	Instalación nueva con 49 funciones recuperadas y 8 ejemplos; sin reportes privados.
database/04_normalizar_catalogos.sql	Migración repetible de la versión anterior. El inicio crea un respaldo antes de aplicarla.
docs/evidencias/normalizacion.json	Recuento y hashes antes/después de migrar.
docs/evidencias/migration-tests.json	9 pruebas nuevas de migración, integridad e instalación.

Compatibilidad verificada

Java consulta los nuevos catálogos con JOIN y conserva los mismos campos JSON. Los filtros, las fichas y AccesI siguen funcionando; Python mantiene su contrato. Se aprobaron 115 pruebas: 20 Java, 43 Python, 39 integración, 4 fallos y 9 de migración e instalación.

Las capturas anteriores de Workbench muestran el esquema de siete tablas antes de esta mejora. El modelo editable y el diagrama actualizado corresponden a nueve tablas. El SQL conserva las reglas CHECK; no se debe reemplazar la base usando el script del compañero sin adaptar la aplicación.

Anexo / diagrama de Workbench

Modelo físico de la versión 1.3: nueve tablas y diez relaciones. La imagen corresponde al modelo editable entregado; sus tablas y columnas se contrastaron con la base MySQL activa.

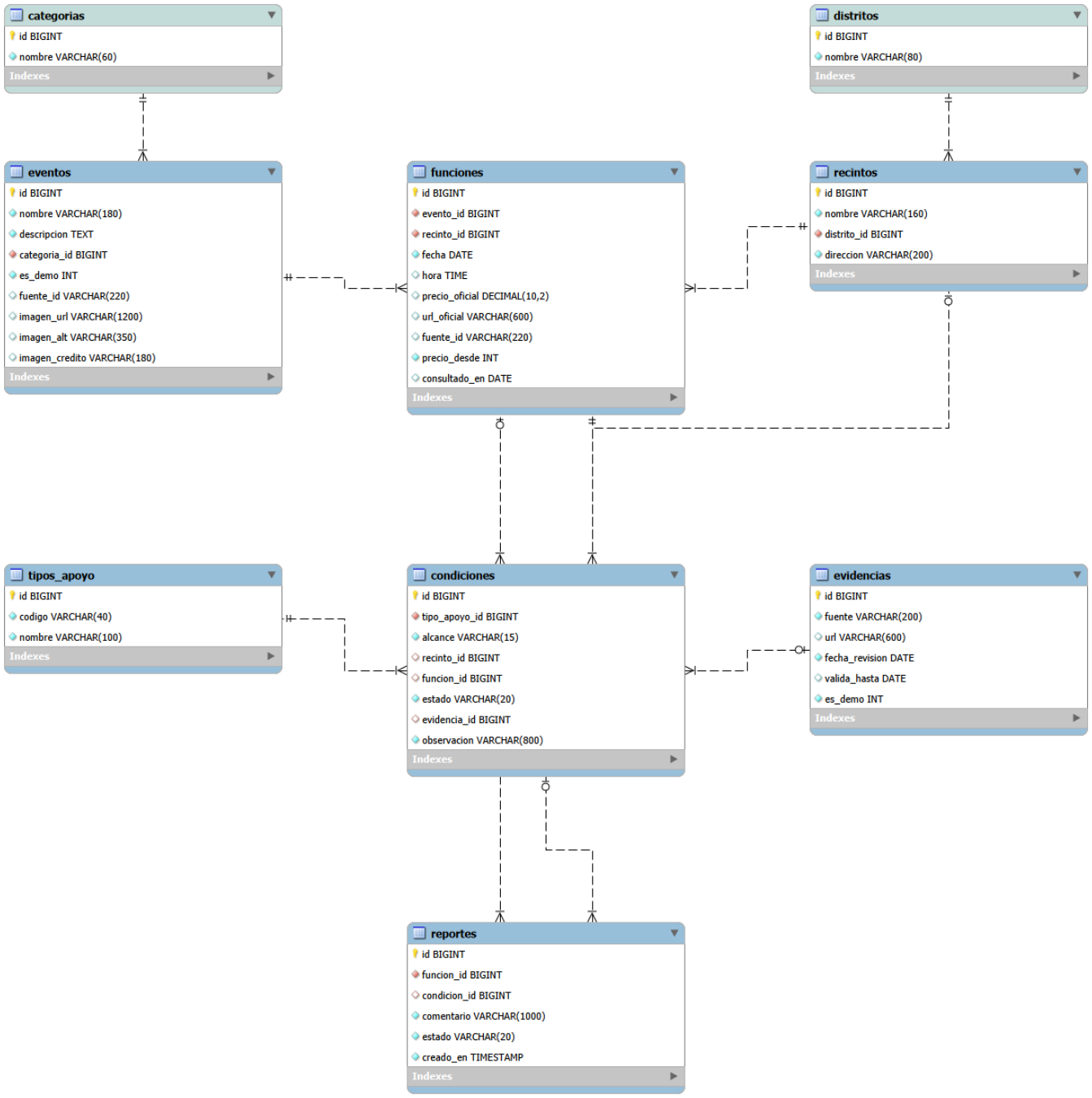


Figura 7. Modelo relacional en MySQL Workbench. Para ampliar los campos, consultar base-datos-workbench.png o CulturaAccesibleLima.mwb, incluidos en docs/.

Mejora aplicada: categorias se relaciona con eventos mediante categoria_id; distritos se relaciona con recintos mediante distrito_id. Se conservan las funciones, las evidencias y los reportes.